MUHAMMAD HASSAAN MUSTAQEEM AWAIS — S2023376018

GitHub: https://github.com/HassaanAwais777/Raye

Raye — Evidence-Based Product Review Sentiment Analysis

Two-page project summary · November 2026

1. Problem and Objective

Product buyers increasingly research through review videos and community discussions, but the signal that matters is buried inside thousands of comments written in English, Urdu, Roman Urdu and free mixtures of all three. Average star ratings hide the reasons behind opinions — price, battery, camera — and video comments carry no rating at all. Raye is deliberately built as a real, working web application rather than a static-dataset exercise: the user types a product name, Raye crawls live comments from popular review sources across multiple platforms, analyzes them multilingually, and returns an evidence-based verdict in which every claim can be traced back to the exact source comment.

2. How an Analysis Runs

A query moves through five stages. First, platform adapters search review videos and threads, then stream comments through rate-limited HTTP clients with exponential-backoff retries. Second, duplicates — identical author, text and platform — are dropped on arrival. Third, every comment passes language identification, spelling normalization, sentiment classification and aspect extraction. Fourth, engagement-weighted aggregation builds per-aspect scorecards and an overall verdict whose confidence value is decomposed into explainable factors. Finally, the dashboard presents the verdict, evidence quotes with deep links, live crawl progress, filters and CSV/JSON export. All results are stored in SQLite and exposed through a documented REST API.

3. Key Capabilities

Requirement	Implementation in Raye
Real-time data (no static datasets)	Live crawlers fetch comments at request time; results cached in SQLite with full job history.
Multi-platform by design	YouTube and Reddit adapters stable; Daraz and Amazon experimental; a registry makes each new platform a one-file plug-in.
English, Urdu, Roman Urdu, code-mixed	Per-comment language identification: Urdu-script detection plus Roman-Urdu marker statistics.
Spelling normalization	A variant map and rules collapse 'bohat', 'bahut', 'bohot', 'bht' and dozens of similar variants into one canonical token before scoring.
Positive / negative / mixed per comment	A transparent lexicon + rules engine with negation windows, intensifiers and contrast-clause splitting; conflicting clauses produce the mixed label.
Aspect-level analysis	Twelve aspects (battery, price, camera, display, performance, software, audio, build, connectivity, delivery, accessories, overall) with multilingual keyword sets.
Traceability to source	Every stored comment keeps the source title, URL and a deep link to the exact comment on the video or thread.

Requirement	Implementation in Raye
Scale to 200,000+ comments	An async fan-out orchestrator with per-platform token-bucket rate limits, deduplication and streaming batch analysis.
Honesty mechanism	Below 150 analyzable comments Raye returns an explicit 'insufficient evidence' verdict with concrete recommendations instead of a fake conclusion.

4. The Multilingual Core

Roman Urdu has no standardized orthography, so the same opinion appears as 'bohat acha', 'bahut acha' or 'bohot acha'. Raye's normalizer collapses these variants — plus texting forms such as 'gr8' and repeated-letter drawls — before scoring, and the raw text is always preserved for display. The sentiment engine then applies a weighted multilingual lexicon covering English, Roman Urdu and Urdu script, negation windows that work in both orders ('achha nahi hai' behaves like 'not good'), intensifiers such as 'bahut', and clause splitting on contrast words like 'lekin' and 'magar', so that praise and criticism in one comment is correctly labelled mixed rather than averaged away.

5. Architecture and Engineering

The system has three layers: a Python FastAPI backend containing the crawlers, the NLP pipeline and the async job orchestrator over SQLite; a React, TypeScript and Vite frontend rendering the verdict dashboard; and a documented REST API between them, deployable with Docker Compose. A pytest suite of 46 tests covers normalization variants, sentiment cases in all four language modes, aspect extraction, the honesty gate and the crawler parsers, and runs on GitHub Actions for every push.

6. Honesty and Traceability

Raye only states a verdict when at least 150 analyzable comments were collected and at least one platform succeeded; otherwise it reports 'insufficient evidence', explains why, and recommends how to gather more data. Even for sufficient data it publishes a confidence score built from sample size, verdict clarity, platform diversity and language coverage, lists warnings such as failed platforms, and marks any aspect with fewer than 30 mentions as unreliable. The three strongest positive, negative and mixed comments are quoted directly, each linking to the exact comment on its source video or thread, so every reader can verify the reasoning.

7. Current Status and Roadmap

The application runs end to end and has been verified with live crawls: real review-video comments — with authors, like counts, dates and working deep links — flow through analysis to a verdict in a single request. Next steps are fine-tuning a transformer model on Roman Urdu to complement the transparent lexicon engine, adding TikTok and X adapters, and weighting comments by predicted helpfulness. Source code, setup instructions and the API reference are available at the GitHub repository listed above.